
Gradient Bitumen

Conversational Sales Agent

Solution Design & Plan

Pure Apex + LWC · LLM-ready · no Agentforce license

A custom Lightning Web Component chat panel (utility bar + Account page) lets a user type what they want; an Apex layer turns that into the full sales chain — Opportunity, Order, Loading Advice and its Line — and auto-submits it for approval. The agent asks for anything missing, confirms before it writes, creates everything in a single all-or-nothing transaction, and sends a confirmation message. Natural-language parsing sits behind a pluggable interface so an LLM can be added later with no rewrite.

Org	salesgvgr — PartialSB sandbox (gb-partialsb), API 66.0
Front-end	Custom LWC chat panel — no Agentforce / no Einstein license
Brains	Apex services + stateless conversation controller; transactional commit
NLU	v1 rule-based parser behind a pluggable interface; LLM-swappable later
Status	Schema confirmed (Step 2 done); ready for Phase 1 build

1. How the agent behaves (the contract)

The agent is single-shot: the user describes the whole job in one message and the agent drives the entire chain. It only pauses to collect missing required fields and to get a go-ahead before writing.

1	User sends one prompt Natural language containing the customer and the deal details, e.g. product, quantity, price, load type. On the Account page the customer is taken from the record; from the utility bar the agent asks for the account name.
2	Agent parses & resolves Extracts values for all four records and resolves the Account (asking which one if the name is ambiguous).
3	Ask for any gaps — once Runs preview across all four records using the curated required-field list. Every missing required field is asked for together in a single follow-up, not one at a time. Loops until complete.
4	Confirm before writing Shows a single summary of everything it will create and waits for an explicit “yes”. (Pre-create gate — see Open Items if you want it removed.)
5	Create — all or nothing Creates Opportunity → Order → Loading Advice → Line in one transaction with a savepoint; any failure rolls the whole thing back so no orphans are left. Sets the LA record type from the load type.
6	Auto-submit for approval Calls the headless approval path (internal PDF + notification + email; status set to Draft / Pending 1st Approver).
7	Confirmation message to the user Returns a success message listing the four created records with links, plus the submission status.

Worked example

User: "Create a commercial loading advice for A A Fugu & Sons – 30 MT of bitumen at NGN 450,000/MT, deliver to Kano."

Agent: "I can set that up. A couple of things first:
 - What close date should I use for the Opportunity?
 - What Order start date?
 - Customer Type for the Loading Advice?"

User: "Close 30/06, order starts today, customer type Reseller."

Agent: "Here is what I will create for A A Fugu & Sons Ltd:
 Opportunity New Business, NGN 13,500,000, close 30/06/2026
 Order starts 01/06/2026, status Draft
 Loading Advice Commercial - 30 MT Bitumen @ 450,000, Kano
 Then auto-submit for approval. Proceed? (yes / no)"

User: "yes"

Agent: "Done. Created Opportunity OPP-1042, Order 00000356, Loading Advice LA-0231 (+ line). Submitted to Joy for approval - PDF, bell alert and email sent."

2. Confirmed ground truth (from Step 2)

These facts were verified against the org via `sf subject describe` and retrieved metadata, and are pinned in `CLAUDE.md` so every session uses them.

Fact	Confirmed value
Opportunity → Account	Lookup (AccountId)
Order → Opportunity / Account	Lookups; AccountId effectively mandatory at insert — agent always sets it
Loading_Advice__c → Order	Master-Detail via Order_DKL__c (non-reparentable, cascade)
Loading_Advice_Line__c → Loading_Advice__c	Master-Detail via LoadingAdvice_DKL__c
Order mandatory at creation	EffectiveDate (start date) + Status (and AccountId in practice)
Load type → Record Type	Commercial → Commercial_DKL; Stock Transfer → StockTransfer_DKL
Dispatch PDF routing	By record type, fires later at approval (LA_Approve_Reject) — NOT part of submit
Header guard	Lock_when_Approved validation rule (relevant only to future update actions)

Required fields = business rule, not the DB

At database level only `Customer_DKL__c` and the master-detail link are non-nillable on the Line. The agent must instead enforce the **curated** required set the Create-LA screen uses: **Customer, Customer Type, Location, Product, Quantity (MT), Price (N/MT)**. This list is the single source of truth for what counts as “missing”.

3. Architecture (pure Apex + LWC, LLM-ready)

Four layers. The UI and the conversation state are the only new build versus the earlier plan; the action library is unchanged and front-end agnostic. Natural-language parsing is isolated behind one interface so it can be upgraded from rule-based to LLM-backed without touching anything else.

Layer	What it is	Notes
1 · Chat UI	LWC <code>gbSalesAgent</code> on the utility bar & Account page	Renders transcript, missing-field prompts, the summary card with Proceed/Cancel, and result links. Holds conversation state client-side.
2 · Conversation controller	Apex <code>@AuraEnabled</code> , stateless	Runs the turn state machine: parse → merge → preview → ask / confirm / commit → submit. Each call receives the accumulated state, so it stays stateless and testable.
3 · Parser (pluggable)	<code>AgentPromptParser</code> interface	<code>v1 RuleBasedPromptParser</code> ; future <code>LLMPromptParser</code> . Active one chosen by Custom Metadata — flip with no code change.
4 · Action services	Plain Apex services (+ thin <code>@InvocableMethod</code> wrappers)	Resolve / insight / describe / preview-chain / commit-chain / submit. Unchanged; reusable by Flow or Agentforce later.

Context & state

- **Account page:** the LWC reads `@api recordId` and uses it as the customer automatically.
- **Utility bar:** no record context, so the agent's first move is to ask for the account name and resolve it (disambiguating if needed).
- **Conversation state** (collected fields, what's been asked, awaiting-confirmation flag, transcript) lives in the LWC and is passed to Apex each turn. This keeps Apex stateless and means the same shape already carries the “history” an LLM will need later.

The LLM seam (future scale)

When you add an LLM, only Layer 3 changes. Build LLMPromptParser to call the model through a **Named Credential** (Salesforce Models API — native and governed — or Anthropic), passing the object/field schema as context and getting back the same structured field map the rule-based parser returns. The LWC, the controller, the transaction logic and every action service stay exactly as built.

Honest scope of v1 parsing

Pure Apex is not natural-language understanding. The rule-based parser reliably extracts structured signals — quantity + unit (30 MT), a price, the load-type keywords, and names it can match against your Account and Product records — and **anything it cannot confidently extract falls through to the ask-for-missing loop**. So v1 behaves as a conversational guided creation with smart pre-fill; it becomes a true free-text agent once the LLM parser is switched on. Set this expectation with the client up front.

Action catalogue (each ships with a test class)

Service / action	Role
AgentAccountService (ResolveAccount)	Resolve customer by recordId or name; disambiguate
AgentInsightService (GetAccountInsight, DescribeObject)	Read-only insight: open opps, order status, recent LAs, required-field explanations (templated in v1; conversational once LLM is on)
AgentChainService.preview (PreviewSalesChain)	Validate all four records from collected fields; return ONE consolidated missingFields list + a single summary
AgentChainService.commit (CommitSalesChain)	With confirmation token: create Opp→Order→LA→Line in one transaction (savepoint + rollback), set LA record type, then submit; return record links
AgentApprovalService (SubmitLAForApproval)	Run the headless approval flow (PDF + notification + email + status)

Shared result wrapper

```
success(Boolean) | status(OK|NEEDS_INPUT|NEEDS_CONFIRM|ERROR) | recordId | recordUrl
missingFields(List<String>) | summaryJson | message
```

4. Transaction & error handling

Because the relationships are master-detail, the children need their parent Id at insert, and a half-built chain would leave orphaned records. The commit is therefore atomic.

- 1 Database.setSavepoint() at the start of **Agent_CommitSalesChain**.
- 2 Insert Opportunity → capture Id → insert Order (with Opp + Account) → capture Id → insert Loading Advice (with Order_DKL__c + RecordTypeId) → capture Id → insert Line (with LoadingAdvice_DKL__c).
- 3 Any DML error → Database.rollback(sp), create nothing, return status **ERROR** with the cause.
- 4 On success → call **Agent_SubmitLAForApproval**, then return all four record links and the submit status.
- 5 Commit refuses to run without a valid confirmation token (tested invariant).

Test coverage required before each deploy

Happy path (4 created + submitted) · each missing-field branch (consolidated ask, nothing created) · commit-without-confirmation rejection · mid-chain failure rolls everything back cleanly. Target ≥90% on new code to keep the org above the 75% gate.

5. Build phases

Phases 1–3 are pure Apex services and are unchanged by the front-end decision. Phases 4–5 build the conversation layer and UI; Phase 6 is the future LLM upgrade, designed-for now and built later.

Phase 1 — Read-only services

- 1 AgentAccountService (resolve), AgentInsightService (insight + describe) + tests; deploy & demo via anonymous Apex.

Phase 2 — Single-prompt chain

- 1 Per-object preview validators using the curated required list.
- 2 AgentChainService.preview (consolidated) and .commit (transactional, sets record type). Full test matrix from §4.

Phase 3 — Headless auto-approval

- 1 Autolaunched flow mirroring ONLY the Submit-for-Approval screen flow; wrap in AgentApprovalService; verify PDF + bell + email.

Phase 4 — Parser + conversation controller

- 1 Define AgentPromptParser + Custom Metadata selector; build RuleBasedPromptParser (regex/units + picklist + Account/Product matching).
- 2 Build the stateless @AuraEnabled conversation controller (the turn state machine) + tests.

Phase 5 — LWC chat UI

- 1 Build gbSalesAgent: transcript, missing-field prompts, summary card with Proceed/Cancel, result links.
- 2 Place on the utility bar (App Manager) and the Account record page (App Builder); wire recordId. End-to-end test from both surfaces.

Phase 6 — LLM parser (future, designed now)

- 1 Add LLMPromptParser via Named Credential (Salesforce Models API or Anthropic); flip the Custom Metadata setting. No change to UI, controller, or actions.

Throughout: each Apex class ships with its test class; keep org coverage ≥75%; FLS/sharing reviewed per service.

6. Open items & assumptions

Item	Default taken	Need from you
Pre-create confirmation gate	Kept (your original brief said 'confirm before creating')	Keep the yes/no gate, or create immediately once fields are complete?
v1 parsing expectation	Rule-based + ask-for-missing fallback (guided creation)	Acknowledged that true free-text NL needs the Phase 6 LLM?
Future LLM provider	Salesforce Models API (native, governed) preferred	Models API, or Anthropic direct via Named Credential?
Existing vs fresh Opp/Order	Always create fresh each run	Reuse an existing open Opportunity/Order if one matches?
Confirmation message channel	In-chat card + record links	Also email the user, or chat only?
Update/delete via agent	Out of v1	Confirm deferred

The matching Claude Code prompt (Claude_Code_Phase2_Prompt.txt) reflects this pure Apex + LWC, LLM-ready design. Phases 1–3 are unchanged from the earlier prompt and proceed as written.